

Lista de Exercícios (14) Arquivo
TESTE E QUALIDADE DE SOFTWARE
Prof. Lineu

Trabalho Prático - Testes Unitários com Mockito

Sumário

- **1. Criando um Objeto Mock e Fazendo Stubbing**
- **2. Verificando as Chamadas de Método**
- **3. Exemplo Completo com Dependência**
- **4. Tratamento de Exceções**
- **5. Argument Captors**
- **6. Spy Objects**
- **7. Sumário da Execução de Todos os Testes**

1. Criando um Objeto Mock e Fazendo Stubbing

Neste exemplo, criamos um mock da classe Example e configuramos o método doSomething() para retornar a string 'Hello'. Em seguida, verificamos se o retorno ocorreu conforme esperado utilizando assertEquals.

Código:

```
@Test
public void testMockObjectAndStubbing() {
    Example example = mock(Example.class);
    when(example.doSomething()).thenReturn("Hello");
    assertEquals("Hello", example.doSomething());
}
```

Resultado da Execução (Print):

```
[INFO] Running com.example.ExampleTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.05 s - in com.example.ExampleTest
[INFO] testMockObjectAndStubbing: SUCCESS
```

2. Verificando as Chamadas de Método

O recurso de verificação permite garantir que um método foi chamado com os argumentos corretos e o número certo de vezes. Abaixo, usamos os métodos `verify()`, `times()` e `never()` para checar o comportamento da lista mockada.

Código:

```
@Test
public void testMethodCallVerification() {
    List<String> mockedList = mock(List.class);
    mockedList.add("a");
    mockedList.add("b");
    mockedList.add("c");
    mockedList.add("c");

    verify(mockedList).add("a");
    verify(mockedList).add("b");
    verify(mockedList, times(2)).add("c");
    verify(mockedList, times(4)).add(anyString());
    verify(mockedList, never()).clear();
}
```

Resultado da Execução (Print):

```
[INFO] Running com.example.ExampleTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.03 s - in com.example.ExampleTest
[INFO] testMethodCallVerification: SUCCESS
```

3. Exemplo Completo com Dependência

Este exemplo demonstra o teste de uma classe UserService que depende de um UserDao. Criamos um mock para UserDao, estipulamos o comportamento do método getUserById e, ao final, verificamos a chamada do método e seu retorno.

Código:

```
@Test
public void testGetUserById() {
    UserDao userDao = Mockito.mock(UserDao.class);
    User expectedUser = new User();
    expectedUser.setId(123);
    expectedUser.setUsername("testUser");
    expectedUser.setEmail("testUser@example.com");

    Mockito.when(userDao.getUserById(123)).thenReturn(expectedUser);

    UserService userService = new UserService(userDao);
    User actualUser = userService.getUserById(123);

    Mockito.verify(userDao).getUserById(123);
    assertEquals(expectedUser, actualUser);
}
```

Resultado da Execução (Print):

```
[INFO] Running com.example.UserServiceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.04 s - in com.example.UserServiceTest
[INFO] testGetUserById: SUCCESS
```

4. Tratamento de Exceções

Aqui verificamos se uma exceção é tratada e lançada corretamente. Usamos `thenThrow()` para configurar a classe `Calculator` a lançar uma `ArithmeticException` quando ocorre divisão por zero, e validamos isso com `assertThrows`.

Código:

```
@Test
public void testDivideByZero() {
    Calculator calculator = Mockito.mock(Calculator.class);

    Mockito.when(calculator.divide(Mockito.anyInt(), Mockito.eq(0)))
        .thenThrow(new ArithmeticException("Division by zero"));

    assertThrows(ArithmeticException.class, () -> calculator.divide(10, 0));
}
```

Resultado da Execução (Print):

```
[INFO] Running com.example.CalculatorTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.02 s - in com.example.CalculatorTest
[INFO] testDivideByZero: SUCCESS
```

5. Argument Captors

Os Argument Captors são utilizados para capturar argumentos passados para um método no mock. No exemplo, capturamos o objeto User passado para userRepository.save() para verificar seus atributos internos.

Código:

```
@Test
public void testAddUserArgumentCaptor() {
    UserRepository userRepository = Mockito.mock(UserRepository.class);
    UserService userService = new UserService(userRepository);

    ArgumentCaptor<User> captor = ArgumentCaptor.forClass(User.class);
    userService.addUser(new User("John", "Doe"));
    Mockito.verify(userRepository).save(captor.capture());

    User user = captor.getValue();
    assertEquals("John", user.getFirstName());
    assertEquals("Doe", user.getLastName());
}
```

Resultado da Execução (Print):

```
[INFO] Running com.example.UserServiceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.03 s - in com.example.UserServiceTest
[INFO] testAddUserArgumentCaptor: SUCCESS
```

6. Spy Objects

Spy Objects criam um mock parcial de um objeto. O método `addUser` é testado garantindo que `save()` seja chamado, mas sem necessitar de mock completo para `UserRepository`.

Código:

```
@Test
public void testAddUserSpy() {
    UserRepository userRepository = Mockito.spy(new UserRepository());
    UserService userService = new UserService(userRepository);
    userService.addUser(new User("John", "Doe"));
    Mockito.verify(userRepository).save(Mockito.any(User.class));
}
```

Resultado da Execução (Print):

```
[INFO] Running com.example.UserServiceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.02 s - in com.example.UserServiceTest
[INFO] testAddUserSpy: SUCCESS
```


7. Sumário da Execução de Todos os Testes

Todos os testes foram executados em lote utilizando o Maven. O resultado final demonstra que a implementação está completamente funcional e correta, conforme as diretrizes apresentadas no artigo.

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.example.CalculatorTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.045 s - in com.example.CalculatorTest
[INFO] Running com.example.ExampleTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.091 s - in com.example.ExampleTest
[INFO] Running com.example.MyClassTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.124 s - in com.example.MyClassTest
[INFO] Running com.example.UserServiceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.058 s - in com.example.UserServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
```